

Documento E — Implementação dos Testes de Integração

Projeto: Sidera Predict

Tecnologia: Flutter

Arquitetura: MVVM com Provider

Tipo de teste: Integração

Testes implementados:

- IT-TC25 — Cadastro e navegação para Menu Principal
- IT-TC26 — Login com campos vazios exibindo validações
- IT-TC27 — Login inválido exibindo mensagem de erro
- IT-TC28 — Login válido navegando para Menu Principal
- IT-TC29 — Fluxo completo: auth, modos visuais, histórico, medição simulada, anotação, IA e histórico final
- TC30 — Salvamento local responsivo mesmo com serviço remoto lento
- TC31 — Otimização de imagem JPEG Full HD sem distorção

Arquivo:

- `integration_test/auth_flow_test.dart`
- `integration_test/full_inspection_flow_test.dart`
- `test/data/measurement_repository_test.dart`

Ferramentas:

- `flutter_test`
- `integration_test`
- `Provider`
- `pumpWidget`
- `pumpAndSettle`
- `tap`
- `enterText`
- `find`
- rota de câmera simulada
- fakes de repositório, storage e IA
- serviço remoto fake com atraso controlado
- imagem sintética para validação de proporção e saída JPEG

Objetivo:

Validar o funcionamento integrado entre Page, Provider, ViewModel, Service fake, navegação, persistência local/remota simulada, estados de IA, confirmação no histórico, salvamento responsivo e otimização segura de imagem.

Resultado esperado:

Os testes devem executar o fluxo de autenticação, simular interações do usuário, verificar validações, mensagens, navegação, alterações de tema, medição simulada, animação de sucesso, anotação, persistência no histórico, transição da IA em `pending` → `generating` → `completed`, retorno rápido em rede lenta e compressão Full HD proporcional.

1. Configuração no pubspec.yaml

```
dev_dependencies:  
  flutter_test:  
    sdk: flutter  
  integration_test:  
    sdk: flutter
```

2. Binding de Integração

Os arquivos de integração inicializam o ambiente com:

```
IntegrationTestWidgetsFlutterBinding.ensureInitialized();
```

3. Aplicação de Teste

Foi criada uma aplicação de teste com:

- `MaterialApp`
- `ChangeNotifierProvider<AuthViewModel>`
- `LoginPage`
- `SignupPage`
- `FakeAuthService`
- Rota fake para `AppRoutes.menuPrincipal`

Essa estrutura permite validar UI e navegação sem usar Supabase real.

Para o fluxo completo (`full_inspection_flow_test.dart`) também foi criada uma aplicação de teste com:

- `MultiProvider`
- `AuthViewModel`
- `InspectionViewModel`
- `SettingsViewModel`
- `MainMenuPage`
- `SettingsPage`
- `HistoryPage`
- `ProcessingPage`
- `ValidationPage`
- rota de câmera simulada
- `MeasurementRepository` com stores e serviços fake
- `OllamaReportService` fake controlado pelo teste

Essa estrutura permite validar o caminho completo sem depender de câmera física, Supabase, Supabase Storage ou Ollama real.

Para os testes de repositório também foram criados:

- `LocalHistoryStore` em memória
- `SupabaseMeasurementService` fake com atraso controlado
- `SupabaseImageStorageService` fake
- `OllamaReportService` fake
- imagem sintética gerada pelo pacote `image`

4. Casos Implementados

IT-TC25 — Cadastro e navegação para Menu Principal

Passos:

1. Abrir tela de login
2. Tocar em "Não tem uma conta? Cadastre-se"
3. Preencher nome
4. Preencher matrícula
5. Preencher e-mail

6. Preencher senha e confirmação
7. Acionar o envio pelo campo "Confirmar Senha"
8. Verificar navegação para "MENU PRINCIPAL"

Resultado esperado:

- Cadastro realizado
- `userName` atualizado
- Tela "MENU PRINCIPAL" exibida

IT-TC26 — Login com campos vazios exibindo validações

Passos:

1. Abrir tela de login
2. Tocar em "ENTRAR" sem preencher campos
3. Verificar mensagens de validação

Resultado esperado:

- Mensagem "Campo obrigatório" exibida
- Mensagem "Senha obrigatória" exibida
- Menu Principal não é aberto

IT-TC27 — Login inválido exibindo mensagem de erro

Passos:

1. Cadastrar usuário no `FakeAuthService`
2. Encerrar sessão fake
3. Abrir tela de login
4. Preencher e-mail correto
5. Preencher senha incorreta
6. Tocar em "ENTRAR"
7. Verificar mensagem de erro

Resultado esperado:

- Mensagem "Credenciais inválidas. Verifique seus dados." exibida
- Menu Principal não é aberto

IT-TC28 — Login válido navegando para Menu Principal

Passos:

1. Cadastrar usuário no `FakeAuthService`
2. Encerrar sessão fake
3. Abrir tela de login
4. Preencher e-mail correto
5. Preencher senha correta
6. Tocar em "ENTRAR"
7. Verificar navegação

Resultado esperado:

- Login realizado
- `userName` atualizado
- Tela "MENU PRINCIPAL" exibida

IT-TC29 — Fluxo completo com modos, medição, anotação e IA

Passos:

1. Preparar usuário no `FakeAuthService`
2. Abrir tela de login
3. Preencher e-mail e senha válidos
4. Confirmar navegação para "MENU PRINCIPAL"
5. Abrir "CONFIGURAÇÕES"
6. Ativar modo escuro
7. Ativar alto contraste
8. Desativar alto contraste e retornar ao modo claro
9. Voltar ao Menu Principal
10. Abrir "HISTÓRICO" e validar estado vazio
11. Voltar ao Menu Principal
12. Abrir "NOVA MEDIÇÃO"
13. Usar a rota "CÂMERA SIMULADA"
14. Acionar "SIMULAR FOTO OK"
15. Processar draft válido pelo `MeasurementService` fake
16. Abrir tela "VALIDAÇÃO"
17. Preencher identificação da peça
18. Marcar "NÃO CONFORME"
19. Selecionar todos os motivos de reprovação
20. Preencher observação/anotação
21. Salvar medição
22. Validar animação "MEDIÇÃO SALVA"

23. Validar registro salvo em fila de IA (`pending`)
24. Validar job IA em execução (`generating`)
25. Liberar relatório final fake
26. Validar IA concluída (`completed`) e relatório preenchido
27. Abrir "HISTÓRICO"
28. Confirmar registro cadastrado com status "NÃO CONFORME"

Resultado esperado:

- Login realizado
- Alterações de modo visual refletidas no `SettingsViewModel`
- Histórico inicialmente vazio
- Medição simulada válida processada
- Animação de sucesso exibida ao salvar
- Identificação, motivo e anotação salvos
- IA passa pelos estados `pending` , `generating` e `completed`
- Registro aparece no histórico final

TC30 — Salvamento local responsivo com nuvem lenta

Passos:

1. Criar `MeasurementRepository` com store em memória.
2. Configurar `SupabaseMeasurementService` fake com atraso controlado.
3. Criar draft válido.
4. Chamar `saveMeasurement` .
5. Medir o tempo de retorno.
6. Verificar que o registro local foi salvo.
7. Verificar que a tentativa remota ocorre posteriormente em segundo plano.

Resultado esperado:

- `saveMeasurement` retorna antes do serviço remoto lento concluir
- Registro fica disponível localmente de imediato
- Sincronização remota é tentada em segundo plano

TC31 — Otimização JPEG Full HD sem distorção

Passos:

1. Criar imagem sintética `4000x2000` .
2. Codificar a imagem original.

3. Chamar `optimizeImageBytesForUpload` .
4. Decodificar a saída.
5. Validar assinatura JPEG.
6. Validar dimensões dentro de `1920x1080` .
7. Validar preservação da proporção.

Resultado esperado:

- Saída em JPEG
- Dimensões otimizadas para `1920x960`
- Proporção original preservada
- Nenhum corte, esticamento ou distorção

5. Execução

Com dispositivo ou emulador conectado:

```
flutter test integration_test
```

Execução dos testes de unidade e integração lógica:

```
flutter test test/ --reporter expanded
```

6. Observação de Ambiente

No ambiente atual, o comando `flutter test integration_test` foi executado com target compatível e retornou sucesso.

Saída observada:

```
00:56 +5: All tests passed!
```

Portanto, os testes de integração foram implementados e executados com sucesso, incluindo o fluxo completo com validação dos estados da IA.